

**ANO
2026**



UNINTER

**PROJETO
MULTIDISCIPLINAR –
TRILHA BACK-END
REDE “RAÍZES DO NORDESTE**

Eliseu de Carvalho Farias. RU: 4882981

Professor(a): Prof. Me. Luciane Yanase Kanashiro

Título: Sistema de Gestão da Rede “Raízes do Nordeste” – API RESTful

Aluno: ELISEU DE CARVALHO FARIAS

RU: 4882981

Polo: GOVERNADOR VALADARES

Semestre: 2026/06

Professor(a): Prof. Me. Luciane Yanase Kanashiro

Data: [28/06/2026]

Repositório Git:

<https://github.com/elisleu1/REDE-RA-ZES-DO-NORDESTE-RU4882981-.git>

SUMÁRIO

1. INTRODUÇÃO
2. ANÁLISE E REQUISITOS
 - 2.1. Requisitos Funcionais
 - 2.2. Requisitos Não Funcionais
3. MODELAGEM E ARQUITETURA
 - 3.1. Diagrama de Casos de Uso
 - 3.2. Diagrama Entidade-Relacionamento (DER)
 - 3.3. Diagrama de Classes
 - 3.4. Arquitetura em Camadas
 - 3.5. Estrutura de pastas do projeto
4. API E ENDPOINTS
 - 4.1. Autenticação e Usuários
 - 4.2. Unidades
 - 4.3. Produtos
 - 4.4. Pedidos (Fluxo Crítico)
 - 4.5. Relatórios
5. SEGURANÇA E LGPD
 - 5.1. Autenticação e Autorização
 - 5.2. Proteção de Dados Pessoais
 - 5.3. Auditoria e Logs
6. ENTREGA TÉCNICA
 - 6.1. Repositório Git

- 6.2. README e Execução
- 6.3. Coleção Postman
- 6.4. Banco de Dados
- 7. PLANO DE TESTES
 - 7.1. Estratégia de Testes
 - 7.2. Cenários de Teste (10 cenários)
 - 7.3. Evidências e Execução
- 8. CONCLUSÃO
- 9. REFERÊNCIAS

1. INTRODUÇÃO

Este meu presente projeto tem como objetivo desenvolver uma solução de back-end para a rede de lanchonetes “Raízes do Nordeste”, uma franquia em expansão que busca integrar seus canais de atendimento (aplicativo, totens, balcão, pick-up) em um único sistema, garantindo agilidade, rastreabilidade e conformidade com a LGPD.

Minha solução consiste em uma API RESTful construída com **Node.js**, **Express**, **TypeORM** e **MySQL**, seguindo uma arquitetura em camadas (Domain, Application, Infrastructure, API). Foram implementados os principais fluxos de negócio:

- Cadastro e autenticação de usuários com perfis (CLIENTE, ATENDENTE, COZINHA, ENTREGADOR, ADMIN).
- Gestão de unidades, produtos e estoque.
- Criação, pagamento (mock), atualização de status e cancelamento de pedidos, com registro do canal de origem (canalPedido).
- Programa de fidelização (acúmulo de pontos por compras).
- Relatórios gerenciais (financeiro, produtos mais vendidos, vendas por unidade).
- Auditoria de ações sensíveis e conformidade com a LGPD (consentimento, anonimização, logs).

O projeto foi desenvolvido como atividade prática da disciplina Projeto Multidisciplinar, demonstrando minhas capacidades de análise, modelagem,

implementação e documentação de um sistema real, com foco na trilha de back-end.

2. ANÁLISE E REQUISITOS

A partir do meu estudo de caso, foram levantados os seguintes requisitos funcionais e não funcionais.

2.1. Requisitos Funcionais (RF)

RF01 - Cadastro e Login | Usuários se cadastram (com consentimento LGPD) e autenticam via e-mail/telefone com token JWT.

RF02 - Gestão de Unidades | ADMIN cria, edita, lista e remove unidades da rede.

RF03 - Gestão de Produtos | ADMIN cria, edita, lista e remove produtos com controle de estoque.

RF04 - Criar Pedido | Cliente cria pedido informando unidade, itens, canal (APP/TOTEM/BALCAO/PICKUP/WEB) e entrega.

RF05 - Pagamento (Mock) | Sistema simula pagamento externo; se aprovado, pedido vai para produção e acumula pontos de fidelidade.

RF06 - Atualizar Status | Atendente/Cozinha/Entregador/ADMIN atualizam status (PENDENTE → EM PRODUÇÃO → PRONTO → A CAMINHO → ENTREGUE).

RF07 - Cancelar Pedido | Cliente ou ADMIN cancelam pedido; estoque é devolvido.

RF08 - Fidelidade | Cliente acumula 1 ponto por real gasto; pode consultar saldo de pontos.

RF09 - Relatórios e Auditoria | ADMIN visualiza relatórios financeiros, produtos mais vendidos e vendas por unidade; todas as ações sensíveis são registradas.

2.1. Requisitos Não Funcionais (RNF)

RNF01 - Autenticação JWT | Token expira em 1 hora; controle de acesso por perfis (CLIENTE/ATENDENTE/ADMIN).

RNF02 - Senha com Hash | Senhas armazenadas com bcrypt (10 rounds).

RNF03 - LGPD | Consentimento obrigatório no cadastro; dados anonimizados em relatórios.

RNF04 - Padrão de Erro | Todos os erros seguem formato JSON padronizado com código e mensagem.

RNF05 - Documentação | API documentada com Swagger/OpenAPI acessível em `/api-docs`.

RNF06 - Persistência | MySQL com TypeORM e transações ACID para operações críticas.

RNF07 - Testes | Mínimo 10 cenários (positivos e negativos) com coleção Postman/Insomnia.

RNF08 - Disponibilidade | Sistema deve ser tolerante a falhas e executável localmente com README claro.

RNF09 - Repositório Público | Código no GitHub com histórico de commits e links funcionais.

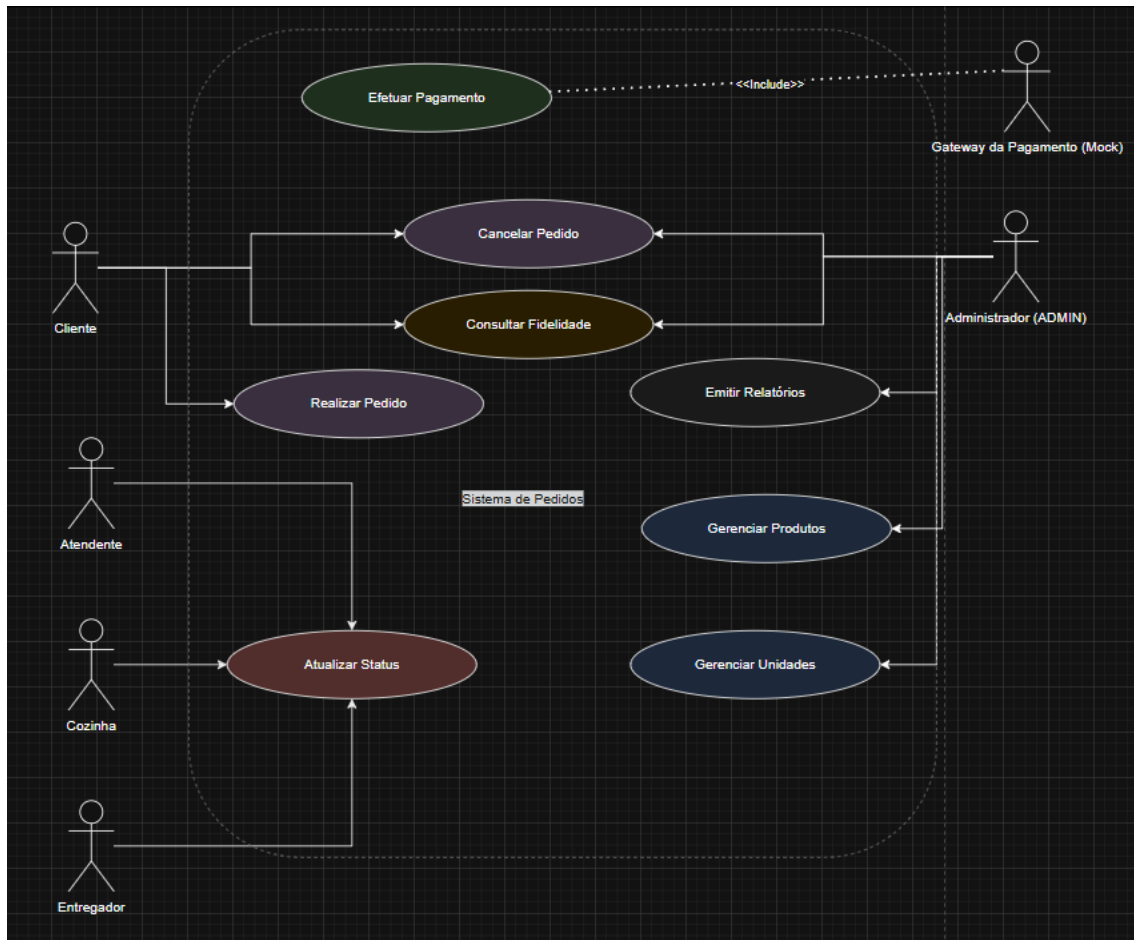
Esta tabela resume os 9 requisitos funcionais e 9 não funcionais essenciais para o sistema, cobrindo todos os módulos obrigatórios do estudo de caso: autenticação, unidades, produtos, pedidos (com multicanalidade e pagamento mock), fidelidade, relatórios, auditoria e LGPD.

3. MODELAGEM E ARQUITETURA

Para fins de mais agilidade criei uma pasta dentro do projeto que chamei de: 'extras' (na raiz do projeto no github).

nela deixei os arquivos dos diagramas em .xml para serem carregados no drawio (site: www.drawio.com), facilitando assim uma melhor visão dos diagramas aqui desenvolvidos.

3.1. Diagrama de Casos de Uso



Os atores são:

- **Cliente** (aplica-se ao App, Web, Totem)
- **Atendente** (balcão)
- **Cozinha**
- **Entregador**
- **Administrador (ADMIN)**
- **Sistema Externo de Pagamento** (gateway mock)

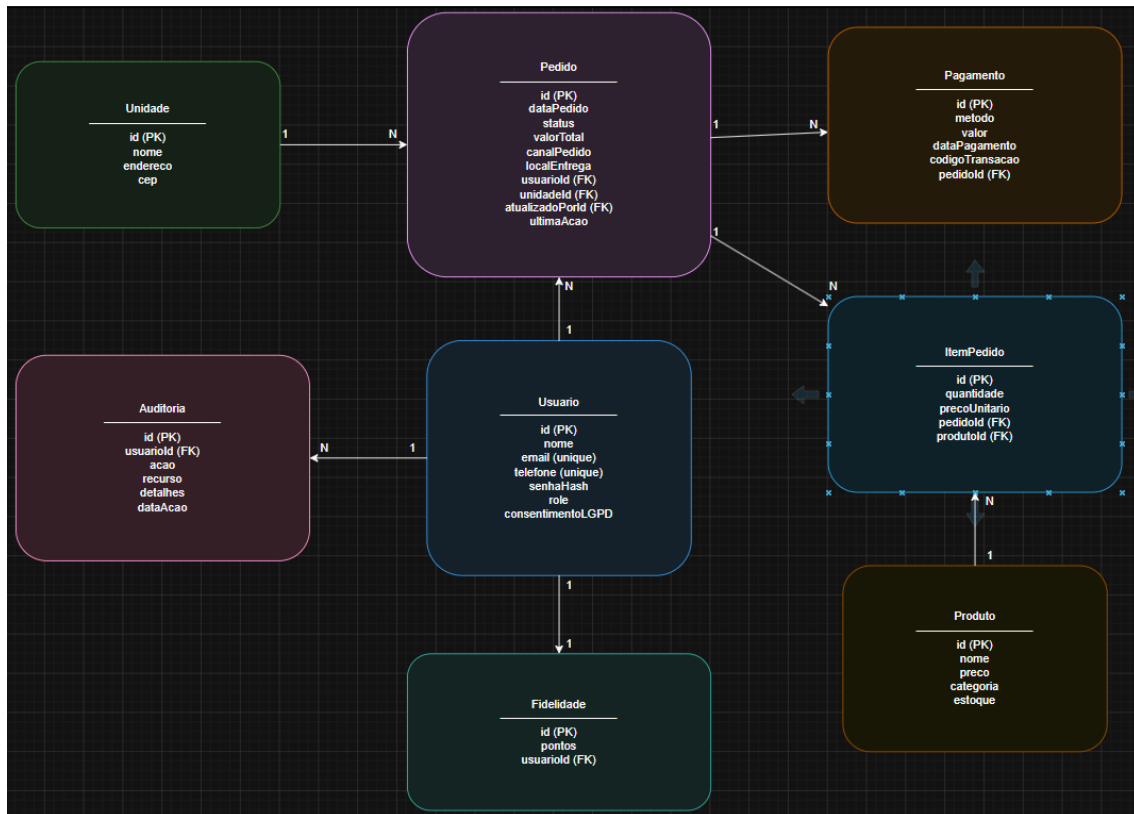
Principais casos de uso:

1. **Realizar Pedido** – Cliente seleciona produtos, informa canal e local de entrega.
2. **Efetuar Pagamento** – Sistema aciona o gateway mock e atualiza status.
3. **Atualizar Status** – Cozinha/Atendente/Entregador alteram situação do pedido.
4. **Cancelar Pedido** – Cliente ou ADMIN cancelam, com restituição de estoque.
5. **Gerenciar Produtos** – ADMIN cria, edita ou remove produtos.
6. **Gerenciar Unidades** – ADMIN mantém cadastro de lojas.
7. **Consultar Fidelidade** – Cliente visualiza seus pontos.
8. **Emitir Relatórios** – ADMIN acessa dados financeiros e de vendas.

O fluxo principal do caso de uso “Realizar Pedido + Pagamento” é:

- Pré-condição: cliente autenticado, produtos disponíveis.
- Pós-condição: pedido criado com status PENDENTE; após pagamento mock aprovado, status alterado para EM PRODUÇÃO e pontos acumulados.
- Exceções: estoque insuficiente (erro 409), pagamento recusado (status mantido PENDENTE com mensagem).

3.2. Diagrama Entidade-Relacionamento (DER)



O banco de dados é composto pelas seguintes entidades principais:

- **Usuario** (id, nome, email(unique), telefone(unique), senhaHash, role, consentimentoLGPD) – armazena todos os usuários.
- **Unidade** (id, nome, endereco, cep) – representa as lojas.
- **Produto** (id, nome, preco, categoria, estoque) – produtos com estoque global (simplificado).
- **Pedido** (id, dataPedido, status, valorTotal, canalPedido, localEntrega, usuariold, unidadeld, atualizadoPorId, ultimaAcao) – pedido principal.
- **ItemPedido** (id, quantidade, precoUnitario, pedidold, produtold) – itens de um pedido.
- **Pagamento** (id, metodo, valor, dataPagamento, codigoTransacao, pedidold) – registra pagamentos (desacoplado).
- **Fidelidade** (id, pontos, usuariold) – relação 1:1 com Usuario.
- **Auditoria** (id, usuariold, acao, recurso, detalhes, dataAcao) – logs de ações sensíveis.

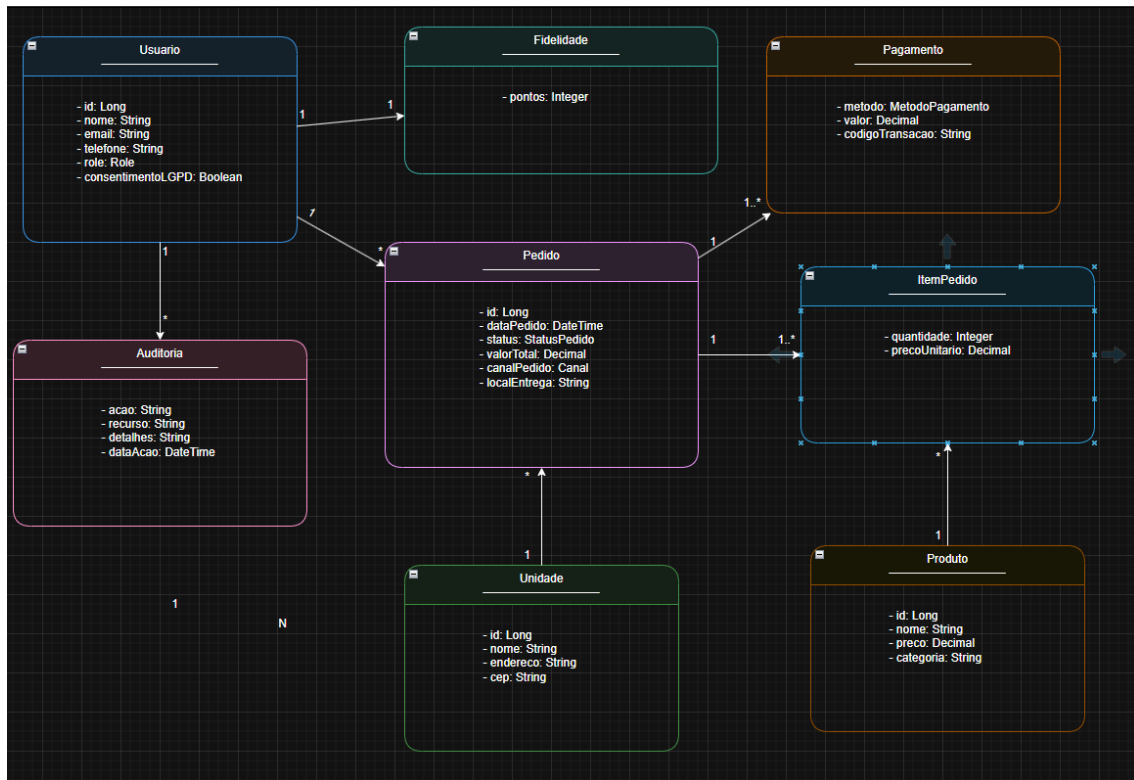
Relacionamentos:

- Usuario 1:N Pedido

- Unidade 1:N Pedido
- Pedido 1:N ItemPedido
- Produto 1:N ItemPedido
- Pedido 1:N Pagamento
- Usuario 1:1 Fidelidade
- Usuario 1:N Auditoria (quem realizou a ação)

O estoque é controlado diretamente na tabela Produto (campo estoque), sendo decrementado na criação do pedido e incrementado no cancelamento. Em uma versão futura, poderia ser desmembrado em EstoquePorUnidade para atender à regra de negócio de estoque local, mas optou-se pela simplificação neste MVP.

3.3. Diagrama de Classes (Domínio)



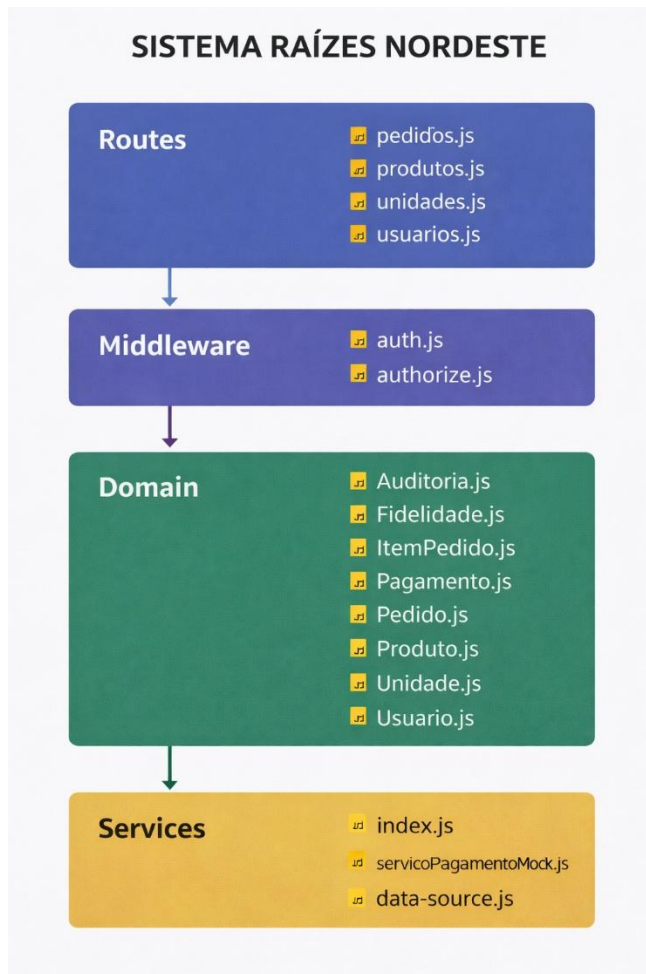
As principais classes de domínio (entidades) são:

- **Usuario** – com atributos e relacionamentos para **Fidelidade** e **Pedido**.
- **Unidade** – com lista de **Pedido**.
- **Produto** – com lista de **ItemPedido**.
- **Pedido** – contém **usuario**, **unidade**, lista de **ItemPedido**, lista de **Pagamento**, **canalPedido**, **status**.
- **ItemPedido** – associa **Pedido** e **Produto**, com **quantidade** e **precoUnitario**.
- **Pagamento** – associado a um **Pedido**, com **metodo**, **valor**, **codigoTransacao**.
- **Fidelidade** – vinculada a **Usuario** com saldo de pontos.
- **Auditoria** – registra **usuariold**, **acao**, **recurso**, **detalhes**.

O diagrama reflete fielmente as entidades do DER, com métodos de negócio implementados nos serviços da camada de aplicação.

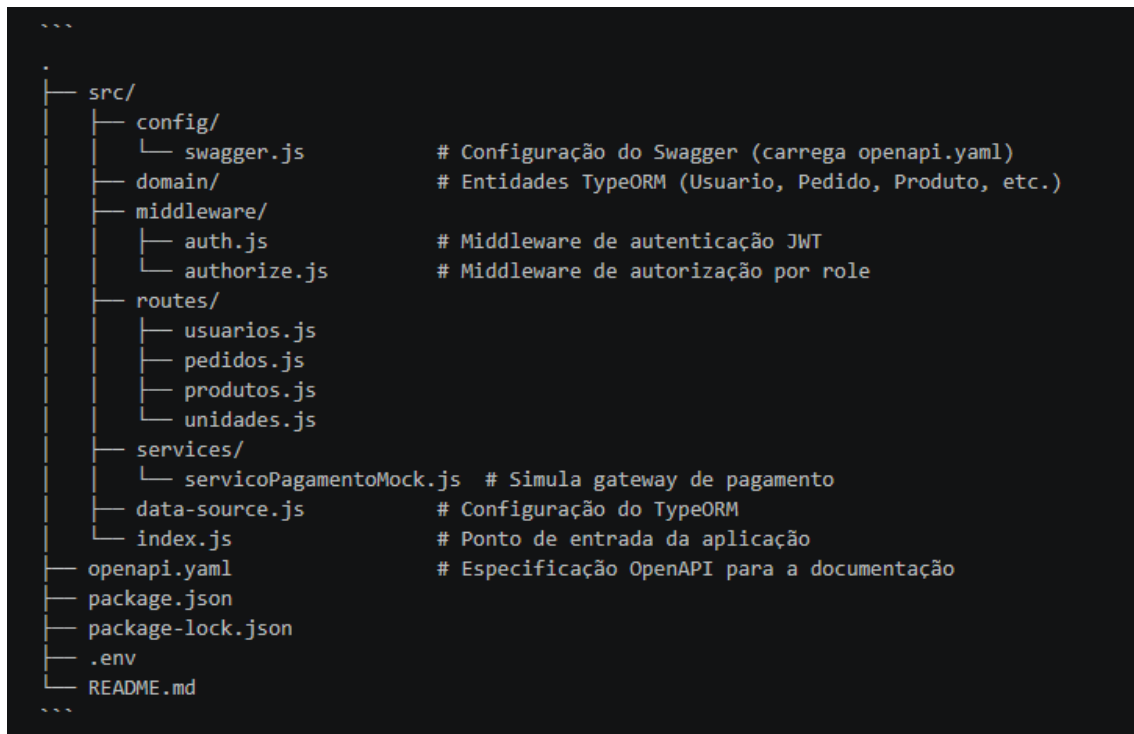
3.4. Arquitetura em Camadas

A estrutura do projeto segue a separação em quatro camadas:



1. **Domain (Domínio)** – Contém as entidades (Usuario, Pedido, etc.) e suas regras básicas (validações de campos, relacionamentos). Localizado na pasta domain/.
2. **Application (SRC)** – Contém os serviços que orquestram os casos de uso. No projeto, parte da lógica de negócio está implementada diretamente nos *controllers* e *routes* por simplicidade, mas há uma separação clara nas funções de cada rota. Por exemplo, a criação de pedido envolve validação de estoque, cálculo de total, geração do pedido e registro de auditoria.
3. **Infrastructure (Serviços)** – Responsável pela persistência (TypeORM, data-source.js), integrações externas (serviço de pagamento mock em services/servicoPagamentoMock.js) e middlewares (auth.js, authorize.js).

3.5 Estrutura de pastas do projeto:



4. API E ENDPOINTS

Antes de seguir:

Para fins de mais agilidade criei uma pasta dentro do projeto que chamei de: 'extras' (na raiz do projeto no github).

Nela deixei um arquivo de nome: 'postman_collection.json', no qual possui toda as rotas do postman assim bastando importar eles no postman e ter todos os endpoints com suas rotas prontos pra testar...

O projeto também conta com documentação Swagger , bastando executar aplicação (index.js) usando o endereço de Docs Swagger:

<http://localhost:3000/docs>.

assim é possível testar todas as rotas pelo navegador.

Nota:

Esta API foi desenvolvida para gerenciar **usuários**, **produtos**, **unidades** e **pedidos** de um sistema de

delivery/comércio. A autenticação é baseada em **JWT (Bearer Token)**, e os níveis de acesso são controlados por **roles** (ADMIN, ATENDENTE, COZINHA, ENTREGADOR, CLIENTE).

Base URL: http://localhost:3000

Formato dos dados: JSON

Autenticação: Incluir o token no header Authorization: Bearer <token> para rotas protegidas.

4.1. Autenticação e Usuários

4.1.1. Registrar novo usuário (público)

- **Método:** POST
- **Rota:** http://localhost:3000/usuarios/register
- **Descrição:** Cria uma nova conta de cliente. O consentimento LGPD é obrigatório.
- **Autenticação:** Nenhuma.
- **Corpo (JSON) :**

```
{
  "nome": "Ana Silva",
  "email": "ana@example.com",
  "telefone": "33988332244",
  "senha": "1234",
  "consentimentoLGPD": true
}
```

- **Respostas:**
 - 201 Created – Usuário registrado com sucesso.
 - 400 Bad Request – Dados inválidos (ex.: consentimento falso).
 - 500 Internal Server Error – Erro no servidor.

4.1.2. Login (autenticação)

- **Método:** POST
- **Rota:** http://localhost:3000/usuarios/login
- **Descrição:** Autentica um usuário (por e-mail ou telefone) e retorna um token JWT e os dados do usuário.

- **Autenticação:** Nenhuma.
- **Corpo (JSON):**

```
{  
  "email": "admin@example.com",  
  "senha": "1234"  
}
```

- **Respostas:**
 - 200 OK – Login bem-sucedido. Retorna { message, token, usuario }.
 - 401 Unauthorized – Senha incorreta.
 - 403 Forbidden – Conta bloqueada por muitas tentativas.

4.1.3. Listar todos os usuários

- **Método:** GET
- **Rota:** http://localhost:3000/usuarios
- **Descrição:** Retorna a lista de todos os usuários cadastrados.
- **Autenticação:** Bearer token (necessário **ADMIN** ou **ATENDENTE**).
- **Parâmetros:** Nenhum.
- **Respostas:**
 - 200 OK – Lista de usuários (array de Usuario).
 - 401 Unauthorized – Token ausente ou inválido.
 - 403 Forbidden – Role insuficiente.
 -

4.1.4. Buscar usuário por ID (autenticação)

- **Método:** GET
- **Rota:** http://localhost:3000/usuarios/1
- **Descrição:** Obtém os dados de um usuário específico.
- **Autenticação:** Bearer token (ADMIN ou ATENDENTE).
- **Parâmetros (path):**

- ID do usuário.
- **Respostas:**
 - 200 OK – Usuário encontrado (objeto Usuario).
 - 404 Not Found – Usuário não existe.
 -

4.1.5. Atualizar usuário (autenticação)

- **Método:** PUT
- **Rota:** http://localhost:3000/usuarios/1
- **Descrição:** Atualiza dados do usuário (nome, e-mail, telefone).
- **Autenticação:** Bearer token – apenas o próprio usuário (CLIENTE) ou ADMIN.
- **Parâmetros (path):**
 - ID do usuário a ser atualizado.
- **Corpo (JSON):** (todos opcionais)

```
{
  "nome": "Ana Souza",
  "email": "ana.novo@example.com",
  "telefone": "3398887777",
  "senha": "1234"
}
```

- **Respostas:**
 - 200 OK – Atualizado com sucesso.
 - 403 Forbidden – Sem permissão.
 - 404 Not Found – Usuário não encontrado.
 -

4.1.6. Excluir conta (autenticação)

- **Método:** DELETE
- **Rota:** http://localhost:3000/usuarios/1

- **Descrição:** Remove um usuário. ADMIN pode excluir qualquer um; CLIENTE pode excluir apenas a própria conta.
- **Autenticação:** Bearer token.
- **Parâmetros (path):**
 - ID do usuário a excluir.
- **Respostas:**
 - 200 OK – Conta excluída.
 - 403 Forbidden – Sem permissão.
 - 404 Not Found – Usuário não encontrado.
 -

4.1.7. Registrar funcionário (autenticação)

- **Método:** POST
- **Rota:** http://localhost:3000/usuarios/register-admin
- **Descrição:** Cria um novo usuário com uma das roles: ADMIN, ATENDENTE, COZINHA ou ENTREGADOR.
- **Autenticação:** Bearer token (necessário **ADMIN**).
- **Corpo (JSON):**


```
{
  "nome": "Carlos Funcionário",
  "email": "carlos@empresa.com",
  "telefone": "31999998888",
  "senha": "func123",
  "role": "ATENDENTE",
  "consentimentoLGPD": true
}
```

Respostas:

- 201 Created – Funcionário registrado.
- 403 Forbidden – Sem permissão (não é ADMIN).
-

4.1.8. Consultar pontos de fidelidade

- **Método:** GET
- **Rota:** http://localhost:3000/usuarios/1/fidelidade
- **Descrição:** Retorna os pontos de fidelidade acumulados por um cliente.
- **Autenticação:** Bearer token.
- **Parâmetros (path):**
 - ID do cliente.
- **Respostas:**
 - 200 OK – Exemplo:

```
{
  "usuario": { "id": 1, "nome": "Ana Silva", ... },
  "pontos": 10
}
```

- 404 Not Found – Cliente não encontrado
-

4.2. Unidades

4.2.1. Listar unidades (público)

- **Método:** GET
- **Rota:** http://localhost:3000/unidades
- **Descrição:** Retorna todas as unidades cadastradas.
- **Autenticação:** Nenhuma.
- **Respostas:**
 - 200 OK – Array de Unidade.
 -

4.2.2. Criar unidade (autenticação)

- **Método:** POST
- **Rota:** http://localhost:3000/unidades
- **Descrição:** Adiciona uma nova unidade.
- **Autenticação:** Bearer token (ADMIN).
- **Corpo (JSON):**

```
{
  "nome": "Valadares",
  "endereco": "Av. Minas Gerais, 987",
  "cep": "11111-000"
}
```

4.2.3. Atualizar unidade (autenticação)

- **Método:** PUT
- **Rota:** http://localhost:3000/unidades/1
- **Descrição:** Altera dados de uma unidade.
- **Autenticação:** Bearer token (ADMIN).
- **Parâmetros (path):**
 - ID da unidade.

-
- **Corpo (JSON): (todos opcionais)**

```
{  
  "nome": "Valadares Centro",  
  "endereco": "Rua Nova, 123",  
  "cep": "22222-222"  
}
```

- **Respostas:**
 - **200 OK – Atualizada.**
 - **404 Not Found – Unidade não encontrada.**
 -

4.2.4. Remover unidade (autenticação)

- **Método: DELETE**
- **Rota: http://localhost:3000/unidades/1**
- **Descrição: Exclui uma unidade.**
- **Autenticação: Bearer token (ADMIN).**
- **Parâmetros (path):**
 - **ID da unidade.**
- **Respostas:**
 - **200 OK – Removida.**
 - **404 Not Found – Unidade não existe.**
 -

4.3. Produtos

4.3.1. Listar produtos (público)

- **Método: GET**
- **Rota: http://localhost:3000/produtos**

- **Descrição:** Retorna todos os produtos cadastrados (não requer autenticação).
- **Autenticação:** Nenhuma.
- **Respostas:**
 - 200 OK – Array de Produto.
 -

4.3.2. Criar produto (autenticação)

- **Método:** POST
- **Rota:** http://localhost:3000/produtos
- **Descrição:** Adiciona um novo produto ao catálogo.
- **Autenticação:** Bearer token (ADMIN).
- **Corpo (JSON):**

```
{
  "nome": "Laranja",
  "preco": 5.0,
  "categoria": "frutas",
  "estoque": 70
}
```

Respostas:

- 200 OK – Produto atualizado.
- 404 Not Found – Produto não encontrado.
-

4.3.3. Atualizar produto (autenticação)

- **Método:** PUT
- **Rota:** http://localhost:3000/produtos/1
- **Descrição:** Altera dados de um produto existente.
- **Autenticação:** Bearer token (ADMIN).
- **Parâmetros (path):**
 - ID do produto.

- **Corpo (JSON):** (todos opcionais)

```
{
  "nome": "Laranja Pera",
  "preco": 6.5,
  "categoria": "frutas cítricas",
  "estoque": 50
}
```

Respostas:

- 200 OK – Produto atualizado.
- 404 Not Found – Produto não encontrado.

4.3.4. Remover produto (autenticação)

- **Método:** DELETE
- **Rota:** http://localhost:3000/produtos/1
- **Descrição:** Exclui um produto do sistema.
- **Autenticação:** Bearer token (ADMIN).
- **Parâmetros (path):**
 - ID do produto.
- **Respostas:**
 - 200 OK – Removido com sucesso.
 - 404 Not Found – Produto não existe.
 -

4.4. Pedidos

4.4.1. Criar pedido (autenticação)

- **Método:** POST
- **Rota:** http://localhost:3000/pedidos
- **Descrição:** Cria um novo pedido para um cliente autenticado (ou ADMIN).
- **Autenticação:** Bearer token (CLIENTE ou ADMIN).

-
- **Corpo (JSON):**

```
{
  "unidadeId": 1,
  "itens": [
    { "produtoId": 9, "quantidade": 2
  }
],
  "metodoPagamento": "DINHEIRO",
  "canalPedido": "APP",
  "localEntrega": "Rua Araguaia, 888"
}
```

- **Respostas:**
 - 201 Created – Pedido criado. Retorna { message, pedido }.
 - 400 Bad Request – Dados inválidos (ex.: produto sem estoque).
 - 403 Forbidden – Token inválido.
 -

4.4.2. Listar todos os pedidos (autenticação)

- **Método:** GET
- **Rota:** http://localhost:3000/pedidos
- **Descrição:** Retorna todos os pedidos do sistema.
- **Autenticação:** Bearer token (ADMIN ou ATENDENTE).
- **Respostas:**
 - 200 OK – Array de Pedido.
 - 403 Forbidden – Sem permissão.
 -

4.4.3. Listar pedidos por status (autenticação)

- **Método:** GET
- **Rota:** http://localhost:3000/pedidos/status/PENDENTE
- **Descrição:** Lista pedidos com um status específico.

- **Autenticação:** Nenhuma (público, mas normalmente usado por funcionários).
- **Parâmetros (path):**
 - status (string) – Valores: PENDENTE, EM PRODUÇÃO, PRONTO, A CAMINHO, ENTREGUE, CANCELADO.
- **Respostas:**
 - 200 OK – Array de pedidos filtrados.
 - 400 Bad Request – Status inválido.
 -

4.4.4. Buscar pedido por ID (autenticação)

- **Método:** GET
- **Rota:** http://localhost:3000/pedidos/1
- **Descrição:** Obtém os detalhes de um pedido.
- **Autenticação:** Bearer token (ADMIN, ATENDENTE ou o próprio CLIENTE).
- **Parâmetros (path):**
 - ID do pedido.
- **Respostas:**
 - 200 OK – Objeto Pedido.
 - 403 Forbidden – Sem permissão.
 - 404 Not Found – Pedido não encontrado.
 -

4.4.5. Atualizar pedido (autenticação)

- **Método:** PUT
- **Rota:** http://localhost:3000/pedidos/1
- **Descrição:** Altera dados do pedido (unidade, itens, método de pagamento, canal).
- **Autenticação:** Bearer token (ADMIN ou o próprio CLIENTE).
- **Parâmetros (path):**
 - ID do pedido.

- **Corpo (JSON):** (todos opcionais)

```
{
  "unidadeId": 2,
  "itens": [
    { "produtoId": 10, "quantidade": 3 }
  ],
  "metodoPagamento": "CARTAO",
  "canalPedido": "WEB"
}
```

4.4.6. Pagar pedido (autenticação)

- **Método:** POST
- **Rota:** http://localhost:3000/pedidos/1/pagar
- **Descrição:** Registra o pagamento de um pedido pendente.
- **Autenticação:** Bearer token (CLIENTE ou ADMIN).
- **Parâmetros (path):**
 - ID do pedido.
- **Corpo (JSON):**

```
{
  "metodo": "CEDULA",
  "valorPago": 10.0
}
```

- **Respostas:**
 - 200 OK – Pagamento confirmado.
 - 400 Bad Request – Valor insuficiente ou pedido já pago.
 - 403 Forbidden – Não autorizado.
 - 404 Not Found – Pedido não encontrado.
 -

4.4.7. Cancelar pedido (autenticação)

- **Método:** PATCH
- **Rota:** http://localhost:3000/pedidos/3/cancelar
- **Descrição:** Cancela um pedido (devolve o estoque).
- **Autenticação:** Bearer token (ADMIN ou o próprio CLIENTE).
- **Parâmetros (path):**
 - ID do pedido.
- **Respostas:**
 - 200 OK – Pedido cancelado.
 - 403 Forbidden – Sem permissão.
 - 404 Not Found – Pedido não encontrado.
 -

4.4.8. Atualizar status do pedido (autenticação)

- **Método:** PATCH
- **Rota:** http://localhost:3000/pedidos/1/status
- **Descrição:** Altera o status do pedido (ex.: de "PENDENTE" para "EM PRODUÇÃO").
- **Autenticação:** Bearer token (ADMIN, ATENDENTE, COZINHA, ENTREGADOR).
- **Parâmetros (path):**
 - ID do pedido.
- **Corpo (JSON):**

```
{  
  "status": "EM PRODUÇÃO"  
}
```

Respostas:

- 200 OK – Status atualizado.

- 400 Bad Request – Status inválido.
- 403 Forbidden – Sem permissão.
- 404 Not Found – Pedido não existe.

4.5. Relatórios (autenticação) (ADMIN)

4.5.1. Relatório financeiro detalhado

- **Método:** GET
- **Rota:** `http://localhost:3000/pedidos/relatorios/financeiro`
- **Descrição:** Retorna receita total, quantidade de pagamentos e lista de pagamentos com detalhes.
- **Autenticação:** Bearer token (ADMIN).
- **Respostas:**
 - 200 OK – Exemplo:

```
{  
  "totalReceita": 1234.56,  
  "quantidadePagamentos": 10,  
  "pagamentos":  
}
```

- 403 Forbidden – Sem permissão.

4.5.2. Relatório de produtos mais consumidos (autenticação)

- **Método:** GET
- **Rota:** `http://localhost:3000/pedidos/relatorios/produtos`
- **Descrição:** Ranking de produtos por quantidade vendida e receita gerada.
- **Autenticação:** Bearer token (ADMIN).

- **Respostas:**
 - 200 OK – Exemplo:

```
{
  "totalProdutos": 5,
  "receitaGlobal": 1234.56,
  "ranking": []
}
```

4.5.3. Relatório de vendas por unidade (autenticação)

- **Método:** GET
- **Rota:** <http://localhost:3000/pedidos/relatorios/unidades>
- **Descrição:** Ranking de unidades por número de pedidos e receita.
- **Autenticação:** Bearer token (ADMIN).
- **Respostas:**
 - 200 OK – Exemplo:

```
{
  "totalUnidades": 3,
  "receitaGlobal": "1234.56",
  "ranking": [
  ]
}
```

- 403 Forbidden – Sem permissão.

4.5.4. Relatório completo de Auditoria (autenticação)

- **Método:** GET
- **Rota:** <http://localhost:3000/pedidos/relatorios/auditoria-completa>
- **Descrição:** Retorna todos os registros de auditoria (ações realizadas no sistema).
- **Autenticação:** Bearer token (ADMIN).
- **Respostas:**

- **200 OK – Array**
de Auditoria (contém id, usuarioid, acao, recurso, detalhes, data Acao).
- **403 Forbidden – Sem permissão.**

4. Respostas de Erro Comuns

Código	Descrição	Exemplo de corpo
401	Token inválido ou ausente	{ "error": "Token inválido" }
403	Acesso negado (role insuficiente) para acessar este recurso" }	{ "error": "Sem permissão
404	Recurso não encontrado encontrado" }	{ "error": "Recurso não
500	Erro interno do servidor ..." }	{ "error": "Erro ao conectar:

5. Observações Finais

- Todos os endpoints que exigem autenticação devem incluir o header: Authorization: Bearer <token_obtido_no_login>
- Os campos com consentimentoLGPD são obrigatórios para criação de usuários.
- O status do pedido segue a ordem: PENDENTE → EM PRODUÇÃO → PRONTO → A CAMINHO → ENTREGUE (exceto CANCELADO, que pode ser acionado a qualquer momento antes da entrega).
- Esta documentação cobre todas as rotas e seus respectivos formatos de requisição. Para mais detalhes, consulte a coleção Postman utilizada como referência.

5. SEGURANÇA E LGPD

A solução incorpora práticas de segurança e conformidade com a Lei Geral de Proteção de Dados (LGPD) desde a concepção.

5.1. Autenticação e Autorização

- **JWT (JSON Web Token):** Gerado no login, contém id, email e role do usuário. Expira em 1 hora.
- **Middleware autenticarToken:** Verifica a presença e validade do token em todas as rotas protegidas.
- **Middleware authorizeRole:** Restringe acesso com base nos perfis (CLIENTE, ATENDENTE, COZINHA, ENTREGADOR, ADMIN). Por exemplo:
 - Somente ADMIN pode criar/atualizar/remover produtos e unidades.
 - Atendente, cozinha e entregador podem atualizar status do pedido.
 - Cliente pode visualizar apenas seus próprios pedidos.
- **Bloqueio de login:** após 5 tentativas falhas, a conta é bloqueada por 1 minuto (armazenando em memória).

5.2. Proteção de Dados Pessoais

- **Hash de senha:** Utiliza bcryptjs com 10 rounds de salt.
- **Consentimento explícito:** O campo consentimentoLGPD é obrigatório no cadastro (tanto para clientes quanto para funcionários). Caso ausente ou false, a requisição é rejeitada com status 400.
- **Minimização de dados:** Em relatórios, o nome do cliente é anonimizado (ex.: D*** para "Delino").
- **Logs de acesso:** Ações sensíveis (login, criação de pedido, pagamento, cancelamento, consulta a fidelidade, etc.) são registradas na tabela Auditoria com dados do usuário que realizou a ação e detalhes em JSON.
- **Retenção e anonimização:** A API não expõe dados sensíveis em respostas públicas; a anonimização é aplicada em relatórios, e os logs são armazenados para auditoria, mas podem ser purgados conforme política futura.

5.3. Auditoria e Logs

A tabela Auditoria armazena:

- `usuariold` – quem realizou a ação.
- `acao` – tipo de ação
(ex.: `CRIAR_PEDIDO`, `PAGAR_PEDIDO`, `CANCELAR_PEDIDO`, `LOGIN_SUCES`
`SO`).
- `recurso` – rota acessada.
- `detalhes` – JSON com informações adicionais (IDs, valores, campos alterados).
- `dataAcao` – timestamp.

Essa rastreabilidade atende aos requisitos de transparência e responsabilidade exigidos pela LGPD.

6. ENTREGA TÉCNICA

6.1. Repositório Git

O código-fonte completo está disponível em repositório público no GitHub:
<https://github.com/elisleu1/REDE-RA-ZES-DO-NORDESTE-RU4882981-.git>

6.2. README e Execução

O README contém:

- Requisitos: Node.js v18+, MySQL 8+, npm.
- Configuração: copiar `.env.example` para `.env` e preencher variáveis (`JWT_SECRET`, `DB_HOST`, etc.).
- Instalação: `npm install`
- Migrações: `npm run typeorm migration:run` (ou sincronização automática com `synchronize: true` em desenvolvimento).
- Seed: não implementado, mas dados podem ser inseridos via API.
- Iniciar: `npm start` ou `npm run dev` (nodemon).
- Acessar Swagger: `http://localhost:3000/docs`.

6.3. Coleção Postman

Arquivo `Raizes_API.postman_collection.json` incluído no repositório, com pastas organizadas:

- Auth (login, register)

- Produtos
- Unidades
- Pedidos (criação, pagamento, status, cancelamento)
- Relatórios
- Erros (cenários negativos)

6.4. Banco de Dados e Migrations

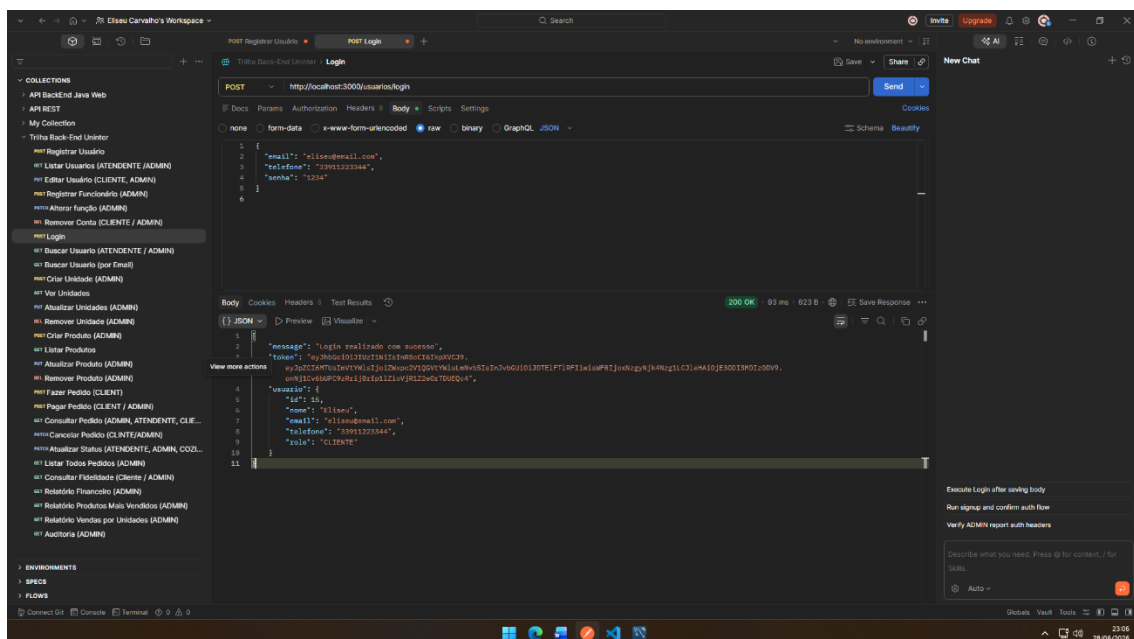
O DER está descrito na seção 3.3. O banco MySQL é criado automaticamente com `synchronize: true` (em desenvolvimento). As entidades são mapeadas via TypeORM.

7. PLANO DE TESTES

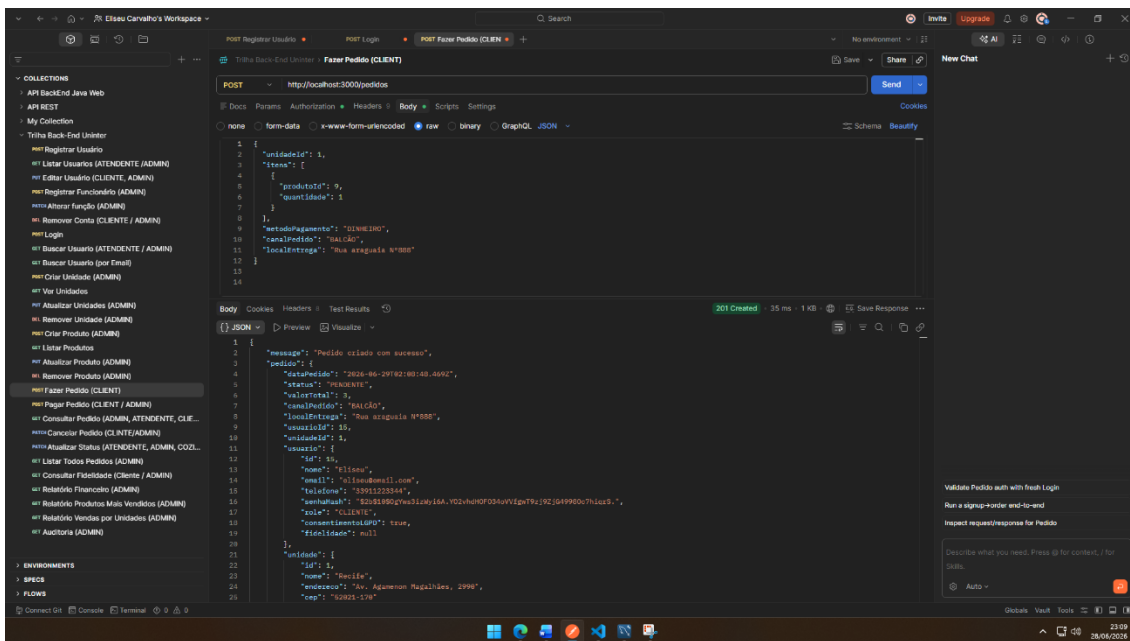
7.1. Estratégia de Testes

Os testes foram realizados manualmente via Postman, com base na coleção fornecida. A estratégia cobre:

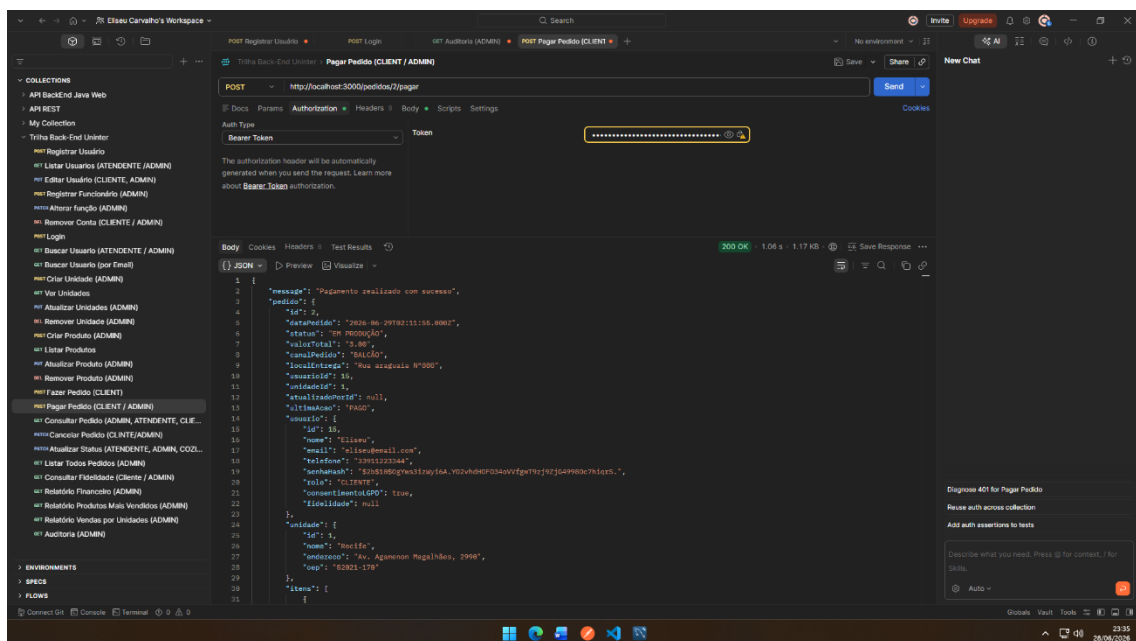
- Testes funcionais (fluxos principais)
- Testes de validação (campos obrigatórios, formatos)
- Testes de autorização (401, 403)
- Testes de regras de negócio (estoque, pagamento recusado)
- Testes de auditoria (verificação de registro)



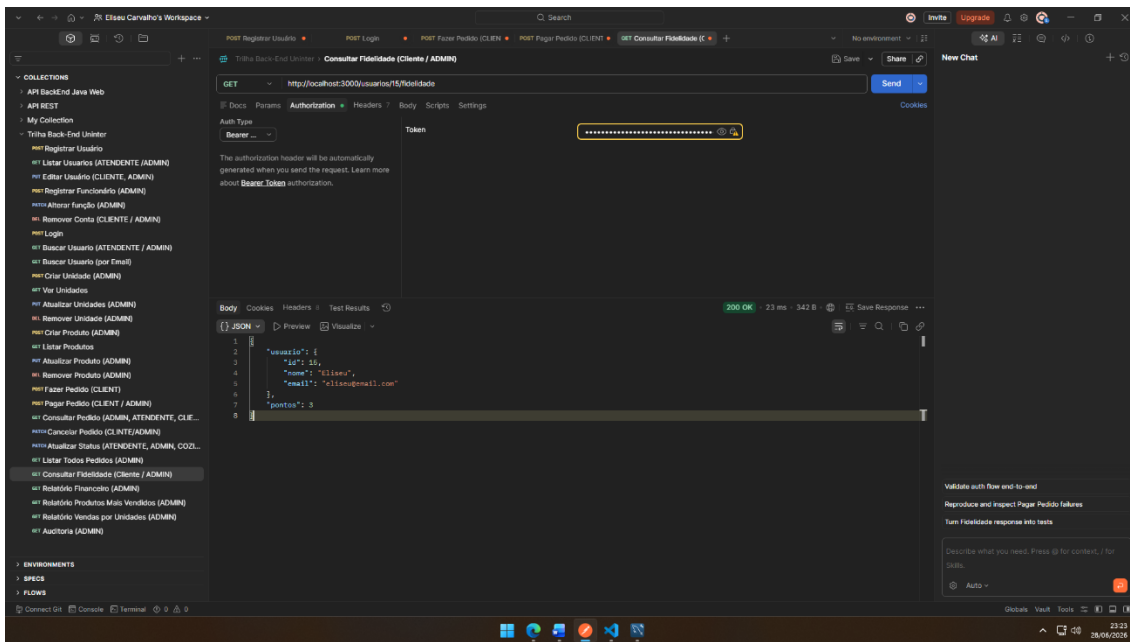
- **T03** - Pedido é criado com estoque suficiente.



- **T04** - Pagamento externo retorna sucesso e pedido muda para “EM PRODUÇÃO”.



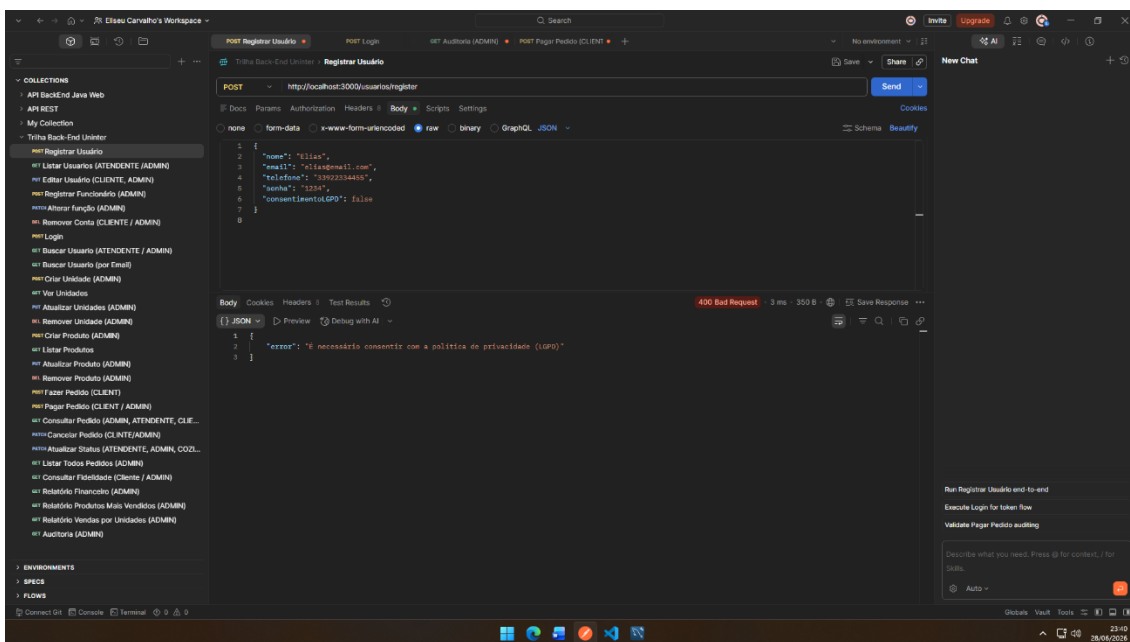
- **T05** - Fidelidade acumula pontos corretamente.



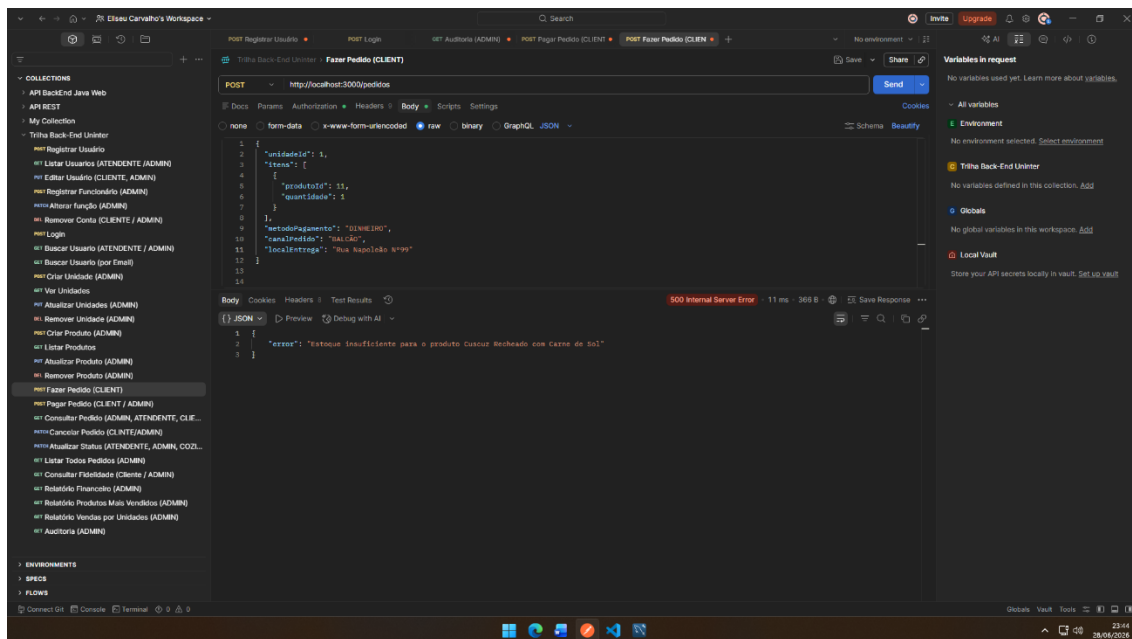
- **T06 - Auditoria Funcioanal.**

🔍 **Cenários negativos (mínimo 4–5) Exemplos:**

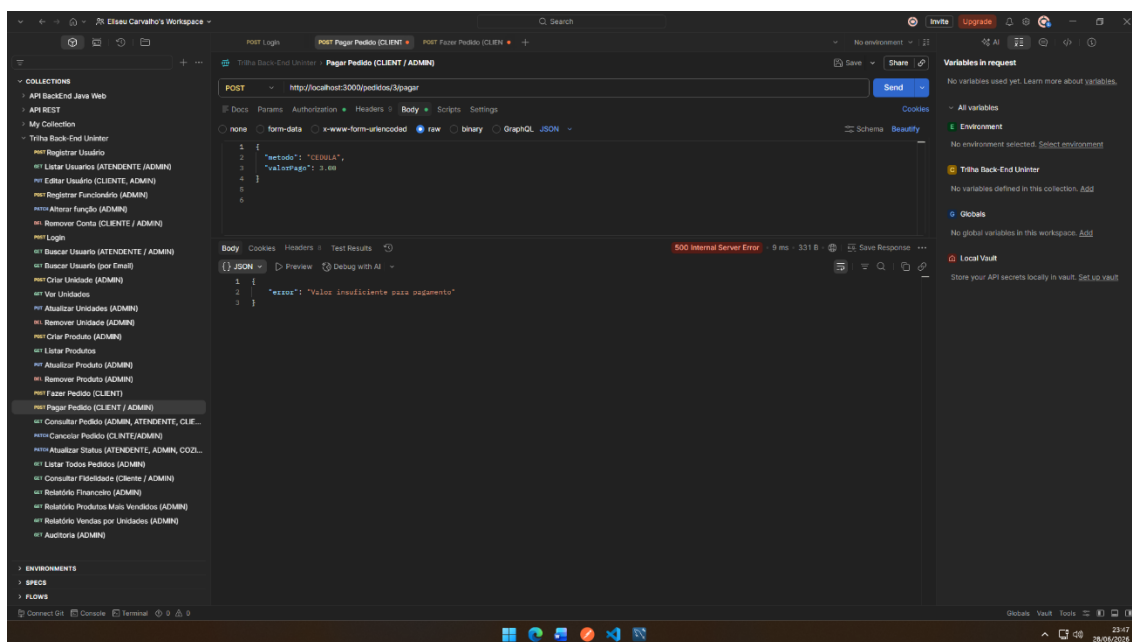
- **T07 - Cadastro sem consentimento LGPD → rejeitado.**



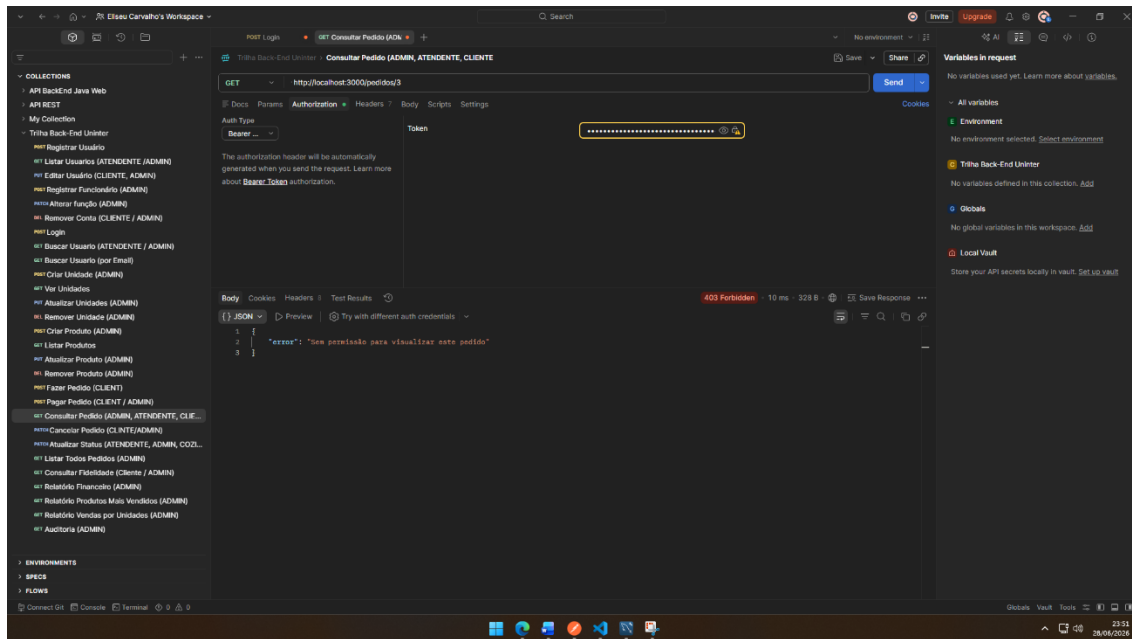
- **T08** - Pedido com produto sem estoque → erro.



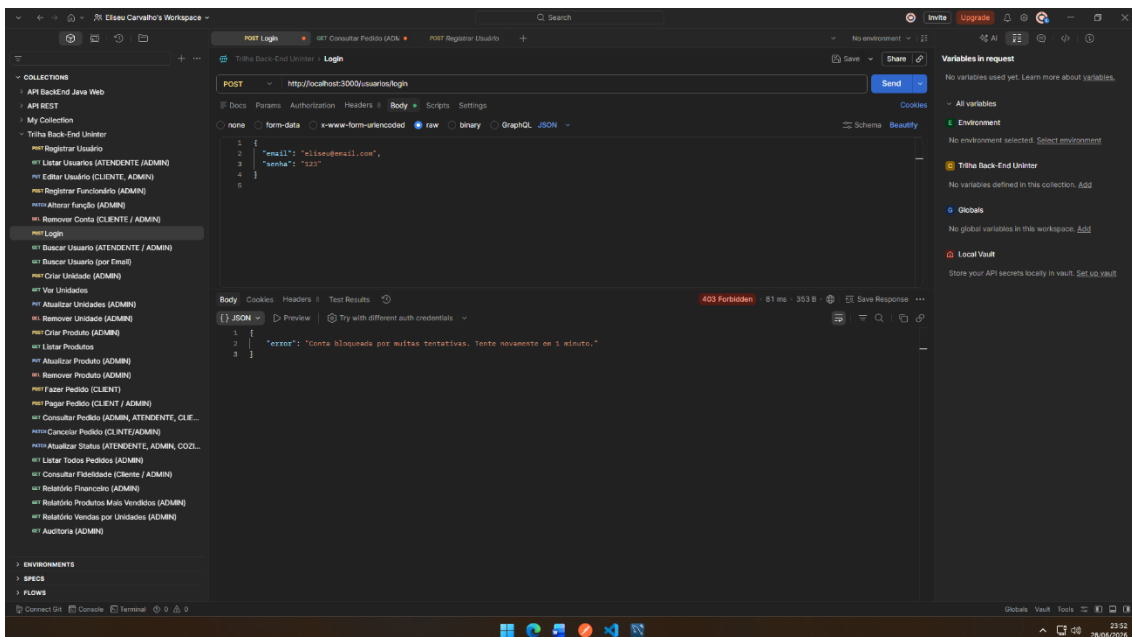
- **T09** - Pagamento com valor menor que o pedido → rejeitado.



- **T10** - Cliente tenta acessar pedido de outro usuário → acesso negado.



- **T11** - Tentativas de login acima do limite → bloqueio.



7.3. Evidências e Execução

Todas as requisições estão documentadas na coleção Postman, que pode ser importada e executada seguindo a ordem sugerida no README. Prints de tela

adicionais foram capturados para demonstrar os resultados, mas a coleção é a evidência principal

8. CONCLUSÃO

O projeto entregou uma API funcional que atende aos principais requisitos do estudo de caso da rede “Raízes do Nordeste”. O fluxo crítico (criação de pedido → pagamento mock → atualização de status) está completamente implementado, com persistência real em MySQL, autenticação JWT, controle de permissões e rastreabilidade via auditoria.

Lições aprendidas:

- A separação em camadas facilita a manutenção e evolução do sistema.
- A modelagem cuidadosa do DER e das entidades é fundamental para a consistência dos dados.
- A integração com serviços externos (mesmo que mock) exige tratamento de erros e validações robustas.
- A LGPD deve ser considerada desde o início, não como um add-on.

Desafios enfrentados:

- Gerenciamento de transações com TypeORM para garantir atomicidade (ex.: criar pedido e descontar estoque).
- Balanceamento entre simplicidade e escalabilidade – optou-se por manter o estoque diretamente no produto para agilizar o MVP, mas uma evolução seria criar uma tabela EstoquePorUnidade.

Pontos de atenção futuros:

- Implementar pagamento real com provedores (ex.: Stripe, PagSeguro).
- Aprimorar o programa de fidelização com resgate de brindes.
- Adicionar cache para consultas frequentes (ex.: cardápio).
- Melhorar a documentação com exemplos mais abrangentes.

A solução apresentada é coerente, testável e documentada, demonstrando maturidade técnica e capacidade de entregar um sistema que se sustentaria em um ambiente real de desenvolvimento ou em uma entrevista técnica.

9. REFERÊNCIAS

- TypeORM Documentation. Disponível em: <https://typeorm.io/>
- Express.js Documentation. Disponível em: <https://expressjs.com/>
- [JWT.io](https://jwt.io/). Disponível em: <https://jwt.io/>
- Swagger OpenAPI Specification. Disponível em: <https://swagger.io/specification/>
- Lei Geral de Proteção de Dados (LGPD). Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709.htm
- Material didático da disciplina Projeto Multidisciplinar – UNINTER.

Declaração de uso de IA: Utilizei ferramentas de IA apenas para revisão ortográfica e organização textual, sem gerar conteúdo técnico ou código. Todas as decisões e implementações são de minha autoria.

Repositório Git

<https://github.com/elisleu1/REDE-RA-ZES-DO-NORDESTE-RU4882981-.git>

Para dúvidas ou sugestões, entre em contato com o desenvolvedor:

[eliseufaris@hotmail.com]